

Regras lógicas e simplificação booleana

Núcleo Cognitivo da Aurora Siger (NCAS)

Atividade Integradora · Fase 5 · Ciência da Computação · FIAP

Grupo 29

Matheus Fuchelberguer · RM571321

Carlos Eugenio Andrade · RM570285

Rodrigo Gomes Dias · RM569142

Implementação: `ncas_core/logica.py`

1. Objetivo

Este documento apresenta as duas regras booleanas que governam as decisões do NCAS, a simplificação algébrica de cada uma e a demonstração de que a forma reduzida preserva o resultado lógico original.

A primeira regra é simplificada pelos teoremas de simplificação; a segunda, pelo primeiro teorema de De Morgan. Ambas estão implementadas na classe MotorLogico, no arquivo `ncas_core/logica.py`, e são verificadas por testes automatizados que comparam a forma original com a simplificada em todas as combinações possíveis de entrada.

Notação adotada: “.” para AND, “+” para OR e “'” para NOT.

2. Regra 1 — Alerta operacional

2.1 Contexto operacional

A colônia registra ocorrências técnicas em seus módulos. Duas variáveis booleanas descrevem cada ocorrência:

- FALHA — indica se foi detectada uma falha no módulo.
- CRITICO — indica se o módulo é essencial (prioridade P1 ou P2).

A regra foi escrita originalmente cobrindo os dois casos de forma explícita: falha em módulo crítico e falha em módulo não crítico.

2.2 Expressão original

```
ALERTA = (FALHA · CRITICO) + (FALHA · CRITICO')
```

2.3 Simplificação passo a passo

Passo 1 — Distributividade do produto sobre a soma (Teorema 16):

$$A \cdot B + A \cdot C = A \cdot (B + C)$$

$$\text{ALERTA} = \text{FALHA} \cdot (\text{CRITICO} + \text{CRITICO}')$$

Passo 2 — Complementaridade da soma (Teorema 4):

$$A + A' = 1$$

$$\text{ALERTA} = \text{FALHA} \cdot 1$$

Passo 3 — Identidade do produto (Teorema 5):

$$A \cdot 1 = A$$

$$\text{ALERTA} = \text{FALHA}$$

2.4 Por que a simplificação mantém o mesmo resultado

Os dois termos da expressão original cobrem os únicos valores que CRITICO pode assumir. Se existe falha, exatamente um dos dois termos é verdadeiro — o primeiro quando CRITICO é verdadeiro, o segundo quando é falso. Como a operação OR exige apenas um termo verdadeiro, a saída é verdadeira em ambos os casos.

Se não existe falha, os dois termos contêm FALHA em uma operação AND e, portanto, ambos são falsos. A saída depende exclusivamente de FALHA: a variável CRITICO é redundante para o ALERTA e pode ser eliminada sem alterar o comportamento do sistema.

2.5 Tabela-verdade — comprovação da equivalência

FALHA	CRITICO	$(F \cdot C) + (F \cdot C')$	FALHA	Comparação
FALSO	FALSO	FALSO	FALSO	iguais
FALSO	VERDADEIRO	FALSO	FALSO	iguais
VERDADEIRO	FALSO	VERDADEIRO	VERDADEIRO	iguais
VERDADEIRO	VERDADEIRO	VERDADEIRO	VERDADEIRO	iguais

As colunas da forma original e da forma simplificada são idênticas nas quatro combinações. A simplificação é, portanto, uma equivalência lógica.

2.6 O que acontece com a variável CRITICO

CRITICO não foi descartado do sistema. Ele deixou de influenciar o ALERTA, mas continua sendo a variável que decide a segunda condição de operação: $EMERGENCIA = FALHA \cdot CRITICO$. Toda falha gera alerta; apenas a falha em módulo essencial aciona o protocolo de contingência imediata.

Essa separação é intencional. Ela evita que o operador seja informado apenas de um valor booleano agregado e permite distinguir uma ocorrência que exige registro de outra que exige ação imediata.

3. Regra 2 — Bloqueio de consulta (De Morgan)

3.1 Contexto operacional

A política de acesso da colônia determina que uma consulta ao NCAS só pode ser liberada quando duas condições ocorrem simultaneamente: o operador está autorizado e o módulo consultado está ativo.

$ACESSO = AUTORIZADO \cdot ATIVO$
 $BLOQUEIO = (AUTORIZADO \cdot ATIVO)'$

3.2 Aplicação do primeiro teorema de De Morgan

O primeiro teorema de De Morgan estabelece que a negação de um produto lógico equivale à soma lógica das negações individuais:

$$(A \cdot B)' = A' + B'$$

Aplicando o teorema à expressão de bloqueio:

$$\text{BLOQUEIO} = \text{AUTORIZADO}' + \text{ATIVO}'$$

3.3 Por que a simplificação mantém o mesmo resultado

A operação AND só produz resultado verdadeiro quando ambas as entradas são verdadeiras. Negar esse resultado significa dizer que ao menos uma das entradas falhou — que é exatamente o que a expressão $\text{AUTORIZADO}' + \text{ATIVO}'$ afirma, já que a operação OR é verdadeira sempre que pelo menos um de seus termos é verdadeiro.

3.4 Tabela-verdade — comprovação da equivalência

AUTORIZADO	ATIVO	$(A \cdot B)'$	$A' + B'$	Comparação
FALSO	FALSO	VERDADEIRO	VERDADEIRO	iguais
FALSO	VERDADEIRO	VERDADEIRO	VERDADEIRO	iguais
VERDADEIRO	FALSO	VERDADEIRO	VERDADEIRO	iguais
VERDADEIRO	VERDADEIRO	FALSO	FALSO	iguais

3.5 Ganho prático da forma negada

Neste caso a simplificação não reduz o número de operações — ela muda a forma da expressão, e é justamente essa mudança que interessa ao sistema.

A forma original produz um único valor booleano: a consulta foi bloqueada ou não. A forma obtida por De Morgan expõe os dois termos que causam o bloqueio separadamente, o que permite ao NCAS avaliar cada um e informar a causa específica ao operador.

AUTORIZADO	ATIVO	Mensagem exibida pelo sistema
VERDADEIRO	VERDADEIRO	Acesso liberado: operador autorizado e módulo ativo.
VERDADEIRO	FALSO	Acesso bloqueado: módulo inativo.
FALSO	VERDADEIRO	Acesso bloqueado: operador não autorizado.
FALSO	FALSO	Acesso bloqueado: operador não autorizado e módulo inativo.

A mensagem é montada pelo método `MotorLogico.motivo_bloqueio()`, que avalia $\text{AUTORIZADO}'$ e ATIVO' de forma independente.

4. Implementação no sistema

As duas regras estão isoladas na classe `MotorLogico`. As formas original e simplificada são mantidas lado a lado de propósito: isso permite que os testes automatizados comprovem a equivalência a cada execução, em vez de a equivalência depender apenas da demonstração em papel.

```
# ncas_core/logica.py

@staticmethod
def calcular_alerta(falha: bool, critico: bool) -> bool:
    """ALERTA = (FALHA . CRITICO) + (FALHA . CRITICO)"""
    return (falha and critico) or (falha and not critico)

@staticmethod
def calcular_alerta_simplificado(falha: bool, critico: bool) -> bool:
    """ALERTA = FALHA"""
    return falha

@staticmethod
def calcular_bloqueio(autorizado: bool, modulo_ativo: bool) -> bool:
    """BLOQUEIO = (AUTORIZADO . ATIVO)"""
    return not (autorizado and modulo_ativo)

@staticmethod
def calcular_bloqueio_simplificado(autorizado, modulo_ativo) -> bool:
    """BLOQUEIO = AUTORIZADO' + ATIVO'"""
    return (not autorizado) or (not modulo_ativo)
```

4.1 Verificação automatizada

Os testes percorrem todas as combinações de entrada e exigem que as duas formas produzam saídas idênticas. Se alguém alterar uma das expressões, o teste falha.

```
def test_forma_simplificada_equivale_a_original(self) -> None:
    for falha in (False, True):
        for critico in (False, True):
            with self.subTest(falha=falha, critico=critico):
                self.assertEqual(
                    MotorLogico.calcular_alerta(falha, critico),
                    MotorLogico.calcular_alerta_simplificado(falha, critico),
                )
```

4.2 Onde as regras são usadas

Regra	Onde é aplicada	Efeito no sistema
ALERTA	Menu 4 e diagnóstico (menu 7)	Marca a ocorrência e a grava nos arquivos
EMERGENCIA	Diagnóstico (menu 7)	Aciona o protocolo de contingência
BLOQUEIO	Menu 5	Libera ou nega a consulta e informa a causa

5. Conclusão

As duas regras foram reduzidas por caminhos diferentes e cada redução foi comprovada por tabela-verdade e por teste automatizado. A primeira eliminou uma variável redundante, tornando a expressão mais curta. A segunda reorganizou a expressão sem encurtá-la, tornando a decisão explicável ao operador.

Em ambos os casos a simplificação preserva integralmente o resultado lógico: nenhuma combinação de entradas produz saída diferente da expressão original.